

P2 — Estrutura de Dados — Material de Estudo com Gabarito

Objetivo: revisar com foco em prova, sem tentar estudar o livro inteiro.

Prioridade: saber escrever algoritmos, explicar complexidade e resolver exemplos manuais.

Como usar este material

Use em 3 passadas:

1. **Primeira passada:** leia os resumos e marque o que você não sabe.
2. **Segunda passada:** resolva os exercícios sem olhar o gabarito.
3. **Terceira passada:** leia apenas o “Kit de Memorização Final”.

0. Aposta de prova

Ordem de prioridade para estudar:

1. **Tabela de dispersão / Hash**
2. **AVL**
3. **Heap / Lista de prioridades**
4. **Ordenação**
5. **Estruturas autoajustáveis**
6. **Árvore binária de busca ótima**
7. **Complexidade e análise amortizada**

1. Kit de Memorização Final

AVL

LL -> rotação simples à direita

RR -> rotação simples à esquerda

LR -> rotação à esquerda no filho, depois à direita no nó

RL -> rotação à direita no filho, depois à esquerda no nó

Heap

$\text{pai}(i) = \lfloor i/2 \rfloor$

$\text{filho esquerdo}(i) = 2i$

$\text{filho direito}(i) = 2i + 1$

inserir -> coloca no fim e sobe

remover raiz -> troca raiz com último, remove e desce

alterar prioridade -> sobe se aumentou, desce se diminuiu

Hash

$h(x)$ = posição da chave

colisão = duas chaves caem na mesma posição

tratamentos:

- encadeamento exterior
- encadeamento interior
- endereçamento aberto

Em endereçamento aberto:

livre

ocupado

removido

Atenção: ao remover, não marque como livre; marque como removido.

Ordenação

Insertion sort: melhor $O(n)$, pior $O(n^2)$

Bubble sort: melhor $O(n)$ com parada, pior $O(n^2)$

Mergesort: $O(n \log n)$

Quicksort: médio $O(n \log n)$, pior $O(n^2)$

Heapsort: $O(n \log n)$

Autoajustáveis

Mover para frente: acessou, joga para o início.

Transposição: acessou, troca com o anterior.

Contador de frequência: mais acessados ficam antes.

Árvore ótima

F -> soma de frequências

C -> custo mínimo

K -> raiz escolhida

Preencher por diagonais.

2. Tabela de dispersão / Hash

2.1 Ideia

Tabela de dispersão é uma estrutura que tenta acessar diretamente uma chave por meio de uma função:

$\text{posição} = h(\text{chave})$

Exemplo:

$$h(x) = x \bmod m$$

Se duas chaves caem na mesma posição, ocorre uma **colisão**.

2.2 Resposta pronta: colisão

Pergunta provável: O que é colisão em tabela de dispersão?

Resposta:

Colisão ocorre quando duas chaves diferentes são mapeadas para a mesma posição da tabela pela função de dispersão. Para resolver colisões, podem ser usados métodos como encadeamento exterior, encadeamento interior ou endereçamento aberto.

2.3 Encadeamento exterior

Cada posição da tabela aponta para uma lista.

```
T[0] -> lista  
T[1] -> lista  
T[2] -> lista  
...
```

Inserção:

```
procedimento inserir(T, x)  
    i := h(x)  
    inserir x na lista T[i]
```

Busca:

```
função buscar(T, x)  
    i := h(x)  
    procurar x na lista T[i]
```

Complexidade:

caso médio: $O(1)$

pior caso: $O(n)$

O pior caso acontece se muitas chaves caírem na mesma posição.

2.4 Endereçamento aberto

Todos os elementos ficam dentro da própria tabela.

Se a posição calculada estiver ocupada, tenta-se outra posição.

Tentativa linear:

$h(x)$, $h(x)+1$, $h(x)+2$, ...

Com módulo:

$(h(x) + i) \bmod m$

2.5 Remoção em endereçamento aberto

Em endereçamento aberto, uma posição pode estar:

livre

ocupada

removida

Pergunta provável: Por que não posso marcar uma posição removida como livre?

Resposta:

Porque isso pode interromper incorretamente uma busca futura. Uma chave que sofreu colisão pode ter sido inserida depois daquela posição. Se a busca encontrar uma posição livre antes da

chave, ela pode concluir erradamente que a chave não existe. Por isso, usa-se o marcador “removido”.

2.6 Exercício de hash

Considere uma tabela de tamanho $m = 7$ e:

$$h(x) = x \bmod 7$$

Insira, usando tentativa linear:

10, 3, 17, 24

Resolução

$$h(10) = 10 \bmod 7 = 3$$

$$h(3) = 3 \bmod 7 = 3$$

$$h(17) = 17 \bmod 7 = 3$$

$$h(24) = 24 \bmod 7 = 3$$

Todos caem na posição 3.

Tabela final:

índice:	0	1	2	3	4	5	6
valor:	-	-	-	10	3	17	24

3. AVL

3.1 Ideia

Uma AVL é uma árvore binária de busca balanceada.

Para cada nó:

$$\text{fator} = \text{altura}(\text{esquerda}) - \text{altura}(\text{direita})$$

Valores permitidos:

-1, 0, +1

Se algum nó fica com fator -2 ou +2, a árvore precisa de rotação.

3.2 Casos de rotação

Caso LL

Inserção na esquerda da esquerda:

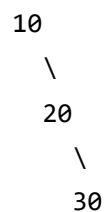
```
      30
     /
    20
   /
  10
```

Rotação simples à direita em 30:

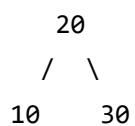
```
      20
     /  \
    10   30
```

Caso RR

Inserção na direita da direita:

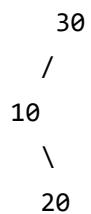


Rotação simples à esquerda em 10:



Caso LR

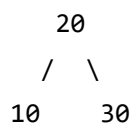
Inserção na direita do filho esquerdo:



Passos:

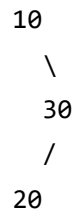
1. rotação à esquerda em 10
2. rotação à direita em 30

Resultado:



Caso RL

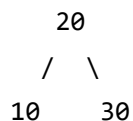
Inserção na esquerda do filho direito:



Passos:

1. rotação à direita em 30
2. rotação à esquerda em 10

Resultado:



3.3 Resposta pronta: por que a rotação funciona?

A inserção em uma AVL só pode alterar a altura dos nós no caminho entre a raiz e o nó inserido. Ao encontrar o primeiro ancestral desbalanceado, uma rotação simples ou dupla reorganiza localmente a subárvore, mantendo a ordem da árvore binária de busca e restaurando o balanceamento. Como a rotação mexe apenas em poucos ponteiros, seu custo é $O(1)$, e a inserção inteira continua $O(\log n)$.

3.4 Complexidades de AVL

busca: $O(\log n)$
inserção: $O(\log n)$
remoção: $O(\log n)$
rotação simples: $O(1)$
rotação dupla: $O(1)$

3.5 Exercício de AVL 1

Insira em uma AVL:

30, 20, 10

Gabarito

Após inserir 30 e 20:

```
  30
 /
20
```

Após inserir 10:

```
    30
   /
  20
 /
10
```

Caso LL. Faz rotação à direita em 30:

```
    20
   / \
  10  30
```

3.6 Exercício de AVL 2

Insira em uma AVL:

10, 30, 20

Gabarito 3.6

Após inserir os três:

```
10
 \
 30
 /
20
```

Caso RL. Primeiro rotação à direita em 30:

```
10
 \
 20
  \
   30
```

Depois rotação à esquerda em 10:

```
  20
 /  \
10    30
```

4. Heap / Lista de prioridades

4.1 Ideia

Lista de prioridades é uma estrutura em que cada elemento possui uma prioridade.

Operações principais:

- consultar o maior
- inserir novo elemento
- remover o maior
- alterar prioridade

Uma implementação eficiente é o **heap**.

4.2 Heap em vetor

Usando índice começando em 1:

- $\text{pai}(i) = \lfloor i/2 \rfloor$
- $\text{filho esquerdo}(i) = 2i$
- $\text{filho direito}(i) = 2i + 1$

Em um max-heap:

- $\text{pai} \geq \text{filhos}$

O maior elemento está na raiz, isto é, na posição 1.

4.3 Inserção em heap

Ideia:

1. coloca no fim
2. compara com o pai
3. enquanto for maior que o pai, troca

Pseudocódigo:

```
procedimento inserir_heap(H, x)
    H.n := H.n + 1
    i := H.n
    H[i] := x

    enquanto i > 1 e H[i] > H[⌊i/2⌋] faça
        trocar(H[i], H[⌊i/2⌋])
        i := ⌊i/2⌋
```

Complexidade:

$O(\log n)$

4.4 Remoção do maior

Ideia:

1. maior está na raiz
2. troca raiz com último
3. diminui o tamanho
4. desce a nova raiz

Pseudocódigo:

```
procedimento remover_max(H)
    se H.n = 0 então
        erro

    max := H[1]
    H[1] := H[H.n]
    H.n := H.n - 1
    descer(H, 1)
    retorna max
```

Complexidade:

$O(\log n)$

4.5 Alterar prioridade

Se a prioridade aumentou:

sobe

Se a prioridade diminuiu:

desce

Resposta pronta:

Ao alterar a prioridade de um elemento, se a nova prioridade for maior, ele pode violar a propriedade de heap em relação ao pai, então deve subir. Se a nova prioridade for menor, ele pode violar a propriedade em relação aos filhos, então deve descer. Em ambos os casos, o custo é $O(\log n)$.

4.6 Construção de heap

Para construir um heap a partir de um vetor:

```
para i := [n/2] até 1 faça  
    descer(i)
```

Complexidade:

$O(n)$

4.7 Heapsort

Ideia:

1. construir heap
2. trocar raiz com último
3. reduzir tamanho do heap
4. descer raiz
5. repetir

Complexidade:

$O(n \log n)$

4.8 Exercício de heap

Dado o max-heap:

índice:	1	2	3	4	5	6
valor:	90	70	80	20	30	60

Insira 85 .

Gabarito 4.8

Coloca 85 no fim:

90, 70, 80, 20, 30, 60, 85

85 está na posição 7. Seu pai está em:

$$\lfloor 7/2 \rfloor = 3$$

Pai = 80. Como $85 > 80$, troca:

90, 70, 85, 20, 30, 60, 80

Agora 85 está na posição 3. Pai:

$$\lfloor 3/2 \rfloor = 1$$

Pai = 90. Como $85 < 90$, para.

Resposta final:

90, 70, 85, 20, 30, 60, 80

5. Ordenação

5.1 Tabela de complexidades

Algoritmo	Melhor caso	Caso médio	Pior caso	Observação
Bubble sort	$O(n)$ com parada	$O(n^2)$	$O(n^2)$	Troca vizinhos
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	Bom para quase ordenado
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Usa vetor auxiliar
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Pivô ruim gera pior caso
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Usa heap

5.2 Insertion sort

Ideia:

A esquerda está ordenada.

Pega o próximo elemento e insere na posição correta.

Pseudocódigo:

```
procedimento insercao(L, n)
  para i := 2 até n faça
    x := L[i]
    j := i - 1

    enquanto j > 0 e L[j] > x faça
      L[j+1] := L[j]
      j := j - 1

    L[j+1] := x
```

Complexidade:

melhor caso: $O(n)$

pior caso: $O(n^2)$

5.3 Mergesort

Ideia:

1. divide a lista ao meio
2. ordena cada metade
3. intercala as duas metades

Recorrência:

$$T(n) = 2T(n/2) + O(n)$$

Resultado:

$O(n \log n)$

5.4 Quicksort

Ideia:

1. escolhe pivô
2. separa menores e maiores
3. ordena os lados recursivamente

Complexidades:

médio: $O(n \log n)$

pior: $O(n^2)$

O pior caso ocorre quando as partições ficam muito desbalanceadas.

5.5 Heapsort

Ideia:

1. construir heap
2. remover o maior repetidamente
3. colocar cada maior no final da tabela

Complexidade:

$O(n \log n)$

5.6 Exercício de ordenação

Dê a complexidade de pior caso:

- a) Insertion sort
- b) Mergesort
- c) Quicksort
- d) Heapsort

Gabarito 5.6

- a) $O(n^2)$
- b) $O(n \log n)$
- c) $O(n^2)$
- d) $O(n \log n)$

Observação:

Quicksort tem caso médio $O(n \log n)$, mas pior caso $O(n^2)$.

6. Estruturas autoajustáveis

6.1 Ideia

Uma estrutura autoajustável reorganiza seus elementos durante as operações.

Resposta pronta:

Estruturas autoajustáveis modificam sua organização conforme são usadas, buscando reduzir o custo de operações futuras. A análise normalmente considera uma sequência de operações, usando a ideia de custo amortizado.

6.2 Mover para frente

Quando um elemento é acessado, ele vai para o início.

Exemplo:

```
lista: A B C D E
acessa D
resultado: D A B C E
```

6.3 Transposição

Quando um elemento é acessado, troca com o anterior.

Exemplo:

```
lista: A B C D E
acessa D
resultado: A B D C E
```

6.4 Contador de frequência

Cada elemento guarda quantas vezes foi acessado.

```
contador := contador + 1
```

Depois o elemento é reposicionado conforme a frequência.

6.5 Custo amortizado

Diferença importante:

pior caso de uma operação:
custo da operação mais cara isolada

custo amortizado:
custo médio por operação em uma sequência

Resposta pronta:

Uma operação isolada pode custar caro, mas, em uma sequência de operações, o custo total pode ser distribuído entre as operações. Esse custo médio por operação é chamado de custo amortizado.

6.6 Exercício de autoajustáveis

Lista inicial:

A B C D E

Acesse D usando:

- a) mover para frente
- b) transposição

Gabarito 6.6

Mover para frente:

D A B C E

Transposição:

A B D C E

7. Árvore binária de busca ótima

7.1 Ideia

Temos chaves ordenadas:

$$x_1, x_2, \dots, x_n$$

Cada chave tem frequência de busca com sucesso:

$$f_1, f_2, \dots, f_n$$

E também há frequências de busca sem sucesso:

$$f'_0, f'_1, \dots, f'_n$$

A árvore ótima é a árvore binária de busca que minimiza o custo esperado das buscas.

7.2 Fórmula do custo

O custo de uma árvore é a soma das frequências multiplicadas pelos níveis/profundidades correspondentes.

Ideia:

chave muito buscada deve ficar mais perto da raiz

Mas atenção: não se escolhe a raiz só “no olho”. O algoritmo testa possibilidades.

7.3 Tabelas usadas

F -> soma das frequências de um intervalo

C -> custo mínimo de um intervalo

K -> raiz escolhida para aquele intervalo

7.4 Fórmulas para memorizar

Soma de frequências:

$$F[i,j] = F[i,j-1] + f[j] + f'[j]$$

Custo mínimo:

$$C[i,j] = \min \{ C[i,k-1] + C[k,j] + F[i,j] \}$$

A raiz k varia dentro do intervalo.

7.5 Como preencher

1. Começa pelos intervalos pequenos.
2. Preenche por diagonais.
3. Para cada intervalo, testa cada possível raiz.
4. Guarda o menor custo em C.
5. Guarda a raiz escolhida em K.

7.6 Exercício conceitual

Explique o papel das tabelas C , F e K .

Gabarito 7.6

F guarda a soma das frequências do intervalo.

C guarda o menor custo possível para aquele intervalo.

K guarda qual chave foi escolhida como raiz daquele intervalo.

8. Complexidade e amortização

8.1 Notação O

Dizer que um algoritmo é $O(n^2)$ significa que seu tempo de execução cresce, no pior caso, no máximo proporcional a n^2 , ignorando constantes e termos menores.

Exemplo:

$$3n^2 + 10n + 8 = O(n^2)$$

8.2 Operações com pilhas

Se temos:

PushA(x)

PushB(x)

PopA()

PopB()

Transfere(k)

Então:

PushA: $O(1)$
PushB: $O(1)$
PopA: $O(1)$
PopB: $O(1)$
Transfere(k): $O(k)$

Se k pode ser maior que a quantidade de elementos de A:

Transfere(k): $O(\min(k, \text{tamanho}(A)))$

No pior caso, se A tem n elementos:

$O(n)$

8.3 Exercício de complexidade

Considere a operação:

Transfere(k): transfere k elementos da pilha A para a pilha B.
Se houver menos de k elementos em A, transfere todos.

Qual a complexidade de pior caso?

Gabarito 8.3

Se A tem n elementos:

$O(\min(k, n))$

No pior caso geral:

$O(n)$

Porque pode ser necessário remover todos os elementos de A e empilhá-los em B.

9. Simulado completo

Resolva sem olhar o gabarito.

Questão 1 — Hash

Considere uma tabela de tamanho $m = 10$ e:

$$h(x) = x \bmod 10$$

Insira usando tentativa linear:

20, 30, 25, 40, 35

Mostre a tabela final.

Questão 2 — AVL

Insira em uma AVL:

50, 40, 30

Mostre a rotação necessária.

Questão 3 — AVL

Insira em uma AVL:

50, 30, 40

Mostre a rotação necessária.

Questão 4 — Heap

Dado o max-heap:

100, 80, 90, 20, 60, 70

Insira 95 .

Questão 5 — Ordenação

Diga a complexidade de pior caso:

- a) Bubble sort
- b) Insertion sort
- c) Mergesort
- d) Quicksort
- e) Heapsort

Questão 6 — Autoajustável

Lista inicial:

10, 20, 30, 40, 50

Acesse 40 .

Mostre o resultado usando:

- a) mover para frente
- b) transposição

Questão 7 — Árvore ótima

Em árvore binária de busca ótima, explique por que uma chave com maior frequência tende a ficar mais perto da raiz, mas por que isso não é suficiente para decidir a árvore ótima.

Questão 8 — Complexidade

Qual a complexidade de `Push`, `Pop` e `Transfere(k)` em pilhas?

10. Gabarito do simulado

Gabarito 1 — Hash

$h(20) = 0$

$h(30) = 0$

$h(25) = 5$

$h(40) = 0$

$h(35) = 5$

Inserindo com tentativa linear:

- 20 vai para 0.
- 30 tenta 0, ocupado; vai para 1.
- 25 vai para 5.
- 40 tenta 0 e 1, ocupados; vai para 2.
- 35 tenta 5, ocupado; vai para 6.

Tabela final:

índice:	0	1	2	3	4	5	6	7	8	9
valor:	20	30	40	-	-	25	35	-	-	-

Gabarito 2 — AVL

Inserção:

50, 40, 30

Árvore antes da rotação:

```
    50
   /
  40
 /
30
```

Caso LL.

Faz rotação simples à direita em 50:

```
    40
   / \
  30  50
```

Gabarito 3 — AVL

Inserção:

50, 30, 40

Árvore antes da rotação:

```
    50
   /
  30
   \
   40
```

Caso LR.

Primeiro, rotação à esquerda em 30:

```
    50
   /
  40
 /
30
```

Depois, rotação à direita em 50:

```
    40
   / \
  30  50
```

Gabarito 4 — Heap

Heap inicial:

100, 80, 90, 20, 60, 70

Insere 95 no final:

100, 80, 90, 20, 60, 70, 95

95 está na posição 7. Pai:

$\lfloor 7/2 \rfloor = 3$

Pai = 90. Como $95 > 90$, troca:

100, 80, 95, 20, 60, 70, 90

Agora 95 está na posição 3. Pai:

$$\lfloor 3/2 \rfloor = 1$$

Pai = 100. Como $95 < 100$, para.

Resposta:

100, 80, 95, 20, 60, 70, 90

Gabarito 5 — Ordenação

- a) Bubble sort: $O(n^2)$
- b) Insertion sort: $O(n^2)$
- c) Mergesort: $O(n \log n)$
- d) Quicksort: $O(n^2)$
- e) Heapsort: $O(n \log n)$

Observação:

Quicksort tem caso médio $O(n \log n)$, mas pior caso $O(n^2)$.

Gabarito 6 — Autoajustável

Lista inicial:

10, 20, 30, 40, 50

Mover para frente:

40, 10, 20, 30, 50

Transposição:

10, 20, 40, 30, 50

Gabarito 7 — Árvore ótima

Resposta:

Uma chave com maior frequência tende a ficar mais perto da raiz porque será buscada mais vezes e, portanto, seu nível influencia bastante o custo total. Porém, isso não é suficiente para decidir a árvore ótima, pois também existem as frequências das outras chaves e das buscas sem sucesso. O algoritmo precisa testar as possíveis raízes de cada intervalo e escolher a combinação que minimiza o custo total.

Gabarito 8 — Complexidade

Push: $O(1)$

Pop: $O(1)$

Transfere(k): $O(k)$

Se a pilha A tem n elementos:

Transfere(k): $O(\min(k, n))$

No pior caso geral:

$O(n)$

11. Última revisão antes da prova

Leia e repita:

AVL:

LL direita

RR esquerda

LR esquerda no filho, direita no nó

RL direita no filho, esquerda no nó

Heap:

inserir sobe

remover desce

alterar prioridade sobe ou desce

Hash:

colisão

encadeamento

endereçamento aberto

removido não é livre

Ordenação:

inserção e bolha $\rightarrow O(n^2)$

merge e heap $\rightarrow O(n \log n)$

quick $\rightarrow$ médio $O(n \log n)$, pior $O(n^2)$

Autoajustáveis:

mover para frente

transposição

frequência

amortizado

Árvore ótima:

F soma

C custo

K raiz

diagonais

12. Estratégia na hora da prova

1. Comece pelas questões conceituais e de complexidade.
2. Em AVL, desenhe a árvore antes da rotação.
3. Em heap, escreva o vetor com índices.
4. Em hash, escreva a tabela com índices.
5. Em árvore ótima, não tente adivinhar raiz: siga a tabela.
6. Em algoritmos, use pseudocódigo simples, no estilo:
 - se ... então
 - enquanto ... faça
 - para ... faça
 - retorna
7. Se travar, escreva a ideia antes do algoritmo. Isso pode render ponto.